



SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

MCQ Test

COURSE NAME: Software Testing

COURSE CODE:CSA3711

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 20%

Code of Conduct:

*I **Manoj A / 192421295** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Answers

Question 1

A team applies systematic test design techniques during development. This reflects:

- a) Debugging
- b) Testing as an engineering activity
- c) Maintenance
- d) Deployment

Answer: b) Testing as an engineering activity

Explanation: Systematic, planned test design is what makes testing an engineering discipline rather than a random activity.

Question 2

If a project skips defined testing processes, what is the likely outcome?

- a) Faster delivery with high quality
- b) Increased defects in production
- c) Reduced testing cost only
- d) Improved documentation

Answer: b) Increased defects in production

Explanation: Without a defined process, coverage is incomplete and defects escape into production.

Question 3

Selecting test cases based on requirements demonstrates:

- a) Random testing
- b) Process-driven testing
- c) Ad hoc testing
- d) Maintenance testing

Answer: b) Process-driven testing

Explanation: Deriving tests from requirements follows a planned process and gives traceability.

Question 4

Applying equivalence partitioning is an example of:

- a) Debugging
- b) Test design technique
- c) Coding standard
- d) Deployment activity

Answer: b) Test design technique

Explanation: Equivalence partitioning is a black-box test design technique.

Question 5

A tester identifies missing requirements during testing. This shows:

- a) Coding error
- b) Process gap
- c) User error
- d) Hardware issue

Answer: b) Process gap

Explanation: Missing requirements point to a gap in the requirement and review process.

Question 6

Following a structured testing lifecycle ensures:

- a) Random defect detection
- b) Improved software quality
- c) Less documentation
- d) No defects

Answer: b) Improved software quality

Explanation: A structured lifecycle improves quality; it cannot guarantee zero defects.

Question 7

Which activity best represents testing as a process?

- a) Writing code
- b) Executing test cases systematically
- c) Deploying software
- d) User training

Answer: b) Executing test cases systematically

Explanation: Systematic execution of planned test cases is the core testing activity.

Question 8

A tester reuses test cases from a repository. This supports:

- a) Efficiency in testing
- b) Increased coding errors
- c) Reduced coverage
- d) Random testing

Answer: a) Efficiency in testing

Explanation: Reuse saves design effort and speeds up testing.

Question 9

If defects are tracked and analyzed, it improves:

- a) Coding speed
- b) Software quality process
- c) Deployment time
- d) User interface

Answer: b) Software quality process

Explanation: Defect analysis reveals root causes and improves the quality process.

Question 10

Applying boundary value analysis helps to:

- a) Increase documentation
- b) Detect edge-case defects
- c) Reduce execution time
- d) Avoid testing

Answer: b) Detect edge-case defects

Explanation: BVA tests values at the edges of ranges, where defects are common.

Question 11

A tester validates system behavior against requirements. This is:

- a) Verification
- b) Validation
- c) Debugging
- d) Coding

Answer: b) Validation

Explanation: Validation checks that the built system meets user needs and requirements.

Question 12

Detecting defects before release reduces:

- a) Development effort
- b) Maintenance cost
- c) Coding standards
- d) Testing scope

Answer: b) Maintenance cost

Explanation: Defects found early cost less to fix than those found in production, reducing maintenance cost.

Question 13

Applying testing principles ensures:

- a) Zero defects guarantee
- b) Early defect detection
- c) No testing needed
- d) Faster coding

Answer: b) Early defect detection

Explanation: Principles such as early testing aim at finding defects sooner; they never guarantee zero defects.

Question 14

Which principle suggests testing all inputs is impossible?

- a) Exhaustive testing
- b) Exhaustive testing is impossible
- c) Early testing
- d) Defect clustering

Answer: b) Exhaustive testing is impossible

Explanation: The principle is named "exhaustive testing is impossible".

Question 15

A tester focuses on frequently failing modules. This uses:

- a) Random testing
- b) Defect clustering principle
- c) Regression testing
- d) Smoke testing

Answer: b) Defect clustering principle

Explanation: Defects cluster in a few modules, so testing is focused there.

Question 16

If testing starts early in SDLC, it helps:

- a) Reduce defects cost
- b) Increase coding errors
- c) Delay project
- d) Skip documentation

Answer: a) Reduce defects cost

Explanation: Early testing finds defects when they are cheapest to fix.

Question 17

A tester designs test cases independently from developers. This ensures:

- a) Bias
- b) Objectivity
- c) Delay
- d) Redundancy

Answer: b) Objectivity

Explanation: Independent testing gives an unbiased, objective view.

Question 18

A tester reviews requirements for errors. Role performed is:

- a) Developer
- b) Tester
- c) Manager
- d) User

Answer: b) Tester

Explanation: Reviewing requirements for errors is a tester's role in early SDLC.

Question 19

Reporting defects clearly is part of:

- a) Coding
- b) Tester's responsibility
- c) Deployment
- d) Maintenance

Answer: b) Tester's responsibility

Explanation: Clear, reproducible defect reports are a tester's responsibility.

Question 20

A tester collaborates with developers to fix bugs. This shows:

- a) Isolation
- b) Teamwork
- c) Delay
- d) Redundancy

Answer: b) Teamwork

Explanation: Collaboration between tester and developer is teamwork.

Question 21

A requirement ambiguity causes defects. Origin is:

- a) Coding
- b) Requirements phase
- c) Testing phase
- d) Deployment

Answer: b) Requirements phase

Explanation: Ambiguity in requirements originates in the requirements phase.

Question 22

Incorrect algorithm implementation leads to:

- a) Requirement defect
- b) Design defect
- c) Coding defect
- d) Testing defect

Answer: c) Coding defect

Explanation: A wrong implementation of the logic is a coding defect.

Question 23

Missing validation checks are example of:

- a) Design defects
- b) Coding defects
- c) Requirement defects
- d) Deployment defects

Answer: b) Coding defects

Explanation: Validation checks are implemented in code, so their absence is a coding defect.

Question 24

Poor UI design causing confusion is:

- a) Coding defect
- b) Design defect
- c) Requirement defect
- d) Hardware defect

Answer: b) Design defect

Explanation: Poor interface layout or flow is a design defect.

Question 25

A system crash due to memory leak is:

- a) Requirement defect
- b) Coding defect
- c) Design defect
- d) Documentation defect

Answer: b) Coding defect

Explanation: A memory leak is caused by improper memory handling in code.

Question 26

Classifying defects based on severity helps:

- a) Ignore defects
- b) Prioritize fixing
- c) Delay release
- d) Increase cost

Answer: b) Prioritize fixing

Explanation: Severity shows impact, so serious defects are fixed first.

Question 27

A critical defect impacts:

- a) Minor feature
- b) Core functionality
- c) UI color
- d) Documentation

Answer: b) Core functionality

Explanation: Critical defects affect core functionality or cause system failure.

Question 28

A minor defect example is:

- a) System crash
- b) Login failure
- c) Typo error
- d) Data loss

Answer: c) Typo error

Explanation: A typo has little effect on functionality, so it is minor.

Question 29

Defect classification helps testers to:

- a) Ignore bugs
- b) Organize defects
- c) Skip testing
- d) Increase errors

Answer: b) Organize defects

Explanation: Classification organises defects for tracking and analysis.

Question 30

A defect repository is used to:

- a) Store code
- b) Store test cases
- c) Store defect details
- d) Store UI designs

Answer: c) Store defect details

Explanation: A defect repository stores all defect information and history.

Question 31

Reusing defect data helps in:

- a) Predicting issues
- b) Increasing bugs
- c) Reducing tests
- d) Skipping phases

Answer: a) Predicting issues

Explanation: Past defect data shows where defects are likely to occur.

Question 32

A defect repository improves:

- a) Code complexity
- b) Testing efficiency
- c) User interface
- d) Deployment speed

Answer: b) Testing efficiency

Explanation: It supports reuse, analysis and tracking, which improve testing efficiency.

Question 33

Logging defects with details ensures:

- a) Confusion
- b) Traceability
- c) Delay
- d) Redundancy

Answer: b) Traceability

Explanation: Detailed records let defects be traced from report to closure.

Question 34

A tester uses past defect data to design tests. This is:

- a) Random testing
- b) Experience-based testing
- c) Debugging
- d) Deployment

Answer: b) Experience-based testing

Explanation: Using past experience and defect history is experience-based testing.

Question 35

Incorrect login validation is an example of:

- a) Functional defect
- b) Performance defect
- c) Security defect
- d) UI defect

Answer: a) Functional defect

Explanation: The login function does not behave as specified, which is a functional defect.

Question 36

Slow response time is a:

- a) Functional defect
- b) Performance defect
- c) UI defect
- d) Coding standard

Answer: b) Performance defect

Explanation: Response time is a performance attribute.

Question 37

Unauthorized data access indicates:

- a) UI defect
- b) Security defect
- c) Functional defect
- d) Design defect

Answer: b) Security defect

Explanation: Access-control failure is a security defect.

Question 38

Missing button functionality is:

- a) Functional defect
- b) Security defect
- c) Performance defect
- d) Hardware defect

Answer: a) Functional defect

Explanation: A feature that does not work as required is a functional defect.

Question 39

Poor alignment in UI is:

- a) Functional defect
- b) UI defect
- c) Security defect
- d) Design defect

Answer: b) UI defect

Explanation: Layout and alignment problems are UI defects.

Question 40

Applying testing principles avoids:

- a) Coding
- b) Redundant tests
- c) Deployment
- d) Maintenance

Answer: b) Redundant tests

Explanation: Principles such as risk-based and clustering-based testing remove wasted, redundant tests.

Question 41

A tester validates outputs using expected results. This is:

- a) Debugging
- b) Test execution
- c) Coding
- d) Maintenance

Answer: b) Test execution

Explanation: Comparing actual and expected results is test execution.

Question 42

If defects are not tracked, it leads to:

- a) Better quality
- b) Poor traceability
- c) Faster delivery
- d) No impact

Answer: b) Poor traceability

Explanation: Untracked defects lose status and ownership, so traceability is poor.

Question 43

Applying test process ensures:

- a) Random results
- b) Structured testing
- c) No defects
- d) No planning

Answer: b) Structured testing

Explanation: A defined test process gives structured, planned testing.

Question 44

Test case design based on requirements ensures:

- a) Low coverage
- b) High coverage
- c) No defects
- d) Less effort

Answer: b) High coverage

Explanation: Requirement-based design covers the specified functionality.

Question 45

Early testing reduces:

- a) Defect cost
- b) Code size
- c) UI design
- d) Deployment time

Answer: a) Defect cost

Explanation: Defects found early cost far less to fix.

Question 46

A tester identifies repeated defects. This indicates:

- a) Good design
- b) Poor process
- c) No issue
- d) Perfect testing

Answer: b) Poor process

Explanation: Recurring defects show weak prevention and process control.

Question 47

Reviewing test results helps:

- a) Ignore defects
- b) Improve quality
- c) Skip testing
- d) Reduce effort

Answer: b) Improve quality

Explanation: Analysis of results guides fixes and improvements.

Question 48

A tester applies regression testing to:

- a) Add features
- b) Ensure fixes work
- c) Deploy software
- d) Design UI

Answer: b) Ensure fixes work

Explanation: Regression testing confirms that changes and fixes did not break existing functionality.

Question 49

Identifying root cause of defects helps:

- a) Increase bugs
- b) Prevent recurrence
- c) Delay testing
- d) Skip coding

Answer: b) Prevent recurrence

Explanation: Fixing the root cause stops the same defect from returning.

Question 50

A test plan ensures:

- a) Random execution
- b) Organized testing
- c) No testing
- d) No defects

Answer: b) Organized testing

Explanation: A test plan defines scope, approach, resources and schedule, so testing is organised.

Question 51

Applying defect classification improves:

- a) Testing cost
- b) Fix prioritization
- c) Coding errors
- d) UI design

Answer: b) Fix prioritization

Explanation: Classification by severity and priority guides the order of fixing.

Question 52

A tester uses historical defect trends. This helps in:

- a) Ignoring defects
- b) Risk-based testing
- c) Deployment
- d) Coding

Answer: b) Risk-based testing

Explanation: Trends show high-risk areas, which is the basis of risk-based testing.

Question 53

Testing ensures product meets:

- a) Developer needs
- b) User requirements
- c) Tester goals
- d) Manager plans

Answer: b) User requirements

Explanation: The purpose of testing is to confirm that the product meets user requirements.

Question 54

A defect due to wrong requirement interpretation is:

- a) Coding defect
- b) Requirement defect
- c) UI defect
- d) Hardware defect

Answer: b) Requirement defect

Explanation: It originates from misunderstanding in the requirements.

Question 55

Applying process standards ensures:

- a) Poor quality
- b) Consistent quality
- c) No testing
- d) Random output

Answer: b) Consistent quality

Explanation: Standards make practice repeatable and results consistent.

Question 56

A tester documents defects clearly. This helps:

- a) Developers fix quickly
- b) Increase errors
- c) Delay release
- d) Reduce quality

Answer: a) Developers fix quickly

Explanation: Clear reports let developers reproduce and fix faster.

Question 57

A defect repository is analyzed to:

- a) Increase cost
- b) Improve testing strategy
- c) Skip testing
- d) Reduce quality

Answer: b) Improve testing strategy

Explanation: Analysis shows patterns that improve test strategy and prevention.

Question 58

A tester applies black-box testing. This focuses on:

- a) Internal code
- b) External behavior
- c) Hardware
- d) Deployment

Answer: b) External behavior

Explanation: Black-box testing checks inputs and outputs without knowledge of the code.

Question 59

Defects found in testing phase indicate:

- a) Perfect design
- b) Earlier phase issues
- c) No issues
- d) Hardware problems

Answer: b) Earlier phase issues

Explanation: Most defects found in testing were introduced in earlier phases such as requirements, design or coding.

Question 60

Applying structured testing improves:

- a) Random results
- b) Product reliability
- c) Coding errors
- d) Deployment delay

Answer: b) Product reliability

Explanation: Structured testing finds more defects and so improves reliability.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	30%	Demonstrates thorough understanding; answers all questions accurately	Shows good understanding with few errors	Partial understanding; several incorrect responses	Lacks understanding; majority of answers incorrect
Accuracy of Answers	25%	All answers are correct	Most answers are correct with minor mistakes	Some correct answers; frequent errors	Mostly incorrect answers
Application of Knowledge	20%	Correctly applies concepts to problem-based questions	Applies concepts with minor errors	Limited ability to apply concepts	Unable to apply concepts
Time Management	15%	Completes quiz well within time efficiently	Completes on time with minor delays	Requires extra time or rushes through questions	Unable to complete within given time
Question Interpretation	10%	Interprets all questions correctly and responds appropriately	Misinterprets a few questions	Often misinterprets questions	Unable to understand questions